

if one rectangle entirely covers another, even if the boundary lines do not intersect. (*Hint*: Move a “sweep” line across the set of rectangles.)

Problems

17-1 *Point of maximum overlap*

You wish to keep track of a *point of maximum overlap* in a set of intervals—a point with the largest number of intervals in the set that overlap it.

- a. Show that there is always a point of maximum overlap that is an endpoint of one of the intervals.
- b. Design a data structure that efficiently supports the operations INTERVAL-INSERT, INTERVAL-DELETE, and FIND-POM, which returns a point of maximum overlap. (*Hint*: Keep a red-black tree of all the endpoints. Associate a value of +1 with each left endpoint, and associate a value of −1 with each right endpoint. Augment each node of the tree with some extra information to maintain the point of maximum overlap.)

17-2 *Josephus permutation*

We define the *Josephus problem* as follows. A group of n people form a circle, and we are given a positive integer $m \leq n$. Beginning with a designated first person, proceed around the circle, removing every m th person. After each person is removed, counting continues around the circle that remains. This process continues until nobody remains in the circle. The order in which the people are removed from the circle defines the *(n, m) -Josephus permutation* of the integers $1, 2, \dots, n$. For example, the $(7, 3)$ -Josephus permutation is $\langle 3, 6, 2, 7, 5, 1, 4 \rangle$.

- a. Suppose that m is a constant. Describe an $O(n)$ -time algorithm that, given an integer n , outputs the (n, m) -Josephus permutation.
- b. Suppose that m is not necessarily a constant. Describe an $O(n \lg n)$ -time algorithm that, given integers n and m , outputs the (n, m) -

Josephus permutation.

Chapter notes

In their book, Preparata and Shamos [364] describe several of the interval trees that appear in the literature, citing work by H. Edelsbrunner (1980) and E. M. McCreight (1981). The book details an interval tree that, given a static database of n intervals, allows us to enumerate all k intervals that overlap a given query interval in $O(k + \lg n)$ time.